Behold, the Perceptron!

DSC 340 Applied Machine Learning

Wednesday, March 19, 2025

The Plan

- Revisit linear models (linear and logistic regression)
- Introduce the perceptron learning algorithm
- Limitations of the model
- Multi-layer perceptrons …

Linear Models

Logistic Regression Revisited

Logistic Regression Revisited

- Logistic regression is a probabilistic model
- Recall that for binary classification, we have:

$$z = \mathbf{w}^\intercal \mathbf{x} + w_0 = w_0 + \sum_{j=1}^{m} w_j x_j$$

$$\mathrm{P}(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-z}}$$

- Weight vector (coefficients) $\mathbf{w}$
- Bias (intercept) w_0

Example in 2D

- Suppose: $\mathbf{w} = (1,1)$, $w_0 = -1$
- Decision boundary is line $x_1 + x_2 = 1$
  - Could rewrite as $x_2 = -x_1 + 1$
  - So slope is -1 and intercept is 1
  - Now we can plot it:

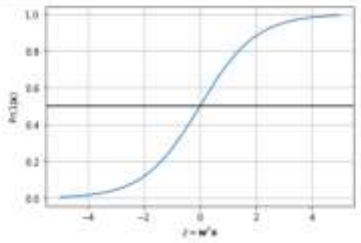
- So where does the probability come in? $\Pr(1|\mathbf{x}) = \frac{1}{1+e^{-(x_1+x_2-1)}}$
  - Suppose we want to classify $(0.5, 0.5)$? Then $z = 0$. Probability = ?
  - You try it: What about (0, 0)? (1, 1)? (0.55, 0.15)?

Shifting the decision boundary

- P(1|x) here is a probability, so it could go anywhere from 0 to 1.
- We saw recently that we could actually vary that.
- What if we set probability threshold at 0.9?
  - We'd have fewer false positives, but also fewer true positives.
- What if we set threshold at 0.1?
  - Then we'd have fewer false negatives, but also a lot of false positives.
- What's the effect geometrically?
  - In our 2D example, just shifts the boundary NW or SE
  - In general, shift the decision boundary along the weight vector $\mathbf{w}$

Logistic Function

In [2]:
```
# Plot the logistic function
z = np.arange(-5., 5., 0.01)
p = expit(z)
plt.plot(z, p)
plt.ylabel('$\Pr(1|\mathbf{x}|$')
plt.xlabel('$z = \mathbf{w}^{T}\mathbf{x}$')
plt.grid(True, which='both')
plt.axhline(y=0.5, color='k')
plt.show()
```



Which side of the boundary?

- Often we could choose a 50/50 threshold. That is:

$$\mathrm{P}(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-z}} = 0.5$$

- So the decision boundary corresponds to a 50/50 chance
- If the new point is on one side, we call it class 1
- If it's on the other, we call if class 0
- And note if the costs are asymmetric, we just shift the boundary
- Make it 80/20 or whatever

Discriminate function

- But when our main objective is classification …
- Often we just ignore the probabilities …
- except to choose the class that is more likely.
- If $P(1|\mathbf{x}) > 0.5$, we call it class 1, else class 0.
- So in fact once we learn the weights:

$$\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1, & \mathbf{w}^{\top}\mathbf{x} + w_0 > 0 \\ 0, & \text{otherwise} \end{cases}$$

- Known as **discriminate function**: "discriminates" (chooses) the class

Big Idea

- If **this** indeed is our model—
- If we only want to do **classification**
- and don't care about the **probabilities**—
- then it occurs that maybe the problem of learning these weights
- could be simpler (computationally and conceptually)
- than estimating probabilities.

Perceptron Learning Algorithm

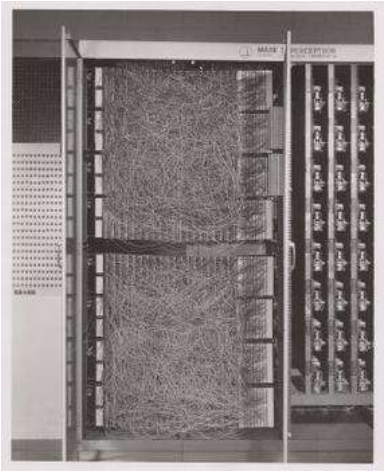
See Assigned Reading and Other Resources on Moodle

Perceptron

- Invented by Frank Rosenblatt of Cornell in 1958, dawn of space age
- Originally supposed to be an intelligent machine
- But discovered it could be simulated with general-purpose computer

Perceptron

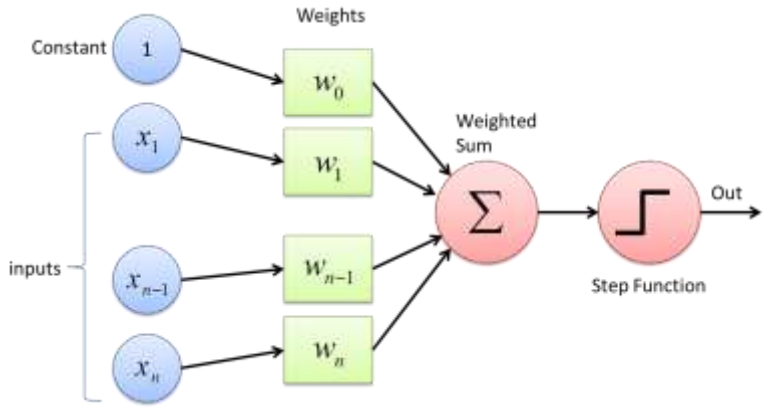
- Later built an actual perceptron machine
- Don't require a lot of computing power; they just need data, loops, and the ability to compute a dot product.
- Nevertheless, could solve interesting problems, created excitement at the time.



BBC Program

- The Machine that Changed The World
- https://www.youtube.com/embed/cNxadbrN_aI

Perceptron


Weights
Constant
1
w_0
x_1
w_1
Weighted
Sum
Out
inputs
x_{n-1}
w_{n-1}
x_n
w_n
Step Function

Perceptron

- Frank Rosenblatt (1957)

$$\hat{\mathbf{y}} = f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Pseudocode

Perceptron learning algorithm

initialize $\mathbf{w} := 0$
repeat:
  for each instance $(\mathbf{x}, y)$ in the training data do:
    if $(\mathbf{x}, y)$ is classified incorrectly then:
      if $f(\mathbf{w} \cdot \mathbf{x}) = 0$, i.e., instance belongs to first class then:
        $\mathbf{w} := \mathbf{w} + \eta\, \mathbf{x}$
      else:
        $\mathbf{w} := \mathbf{w} - \eta\, \mathbf{x}$
until all instances are classified correctly
return $\mathbf{w}$

What can we learn?

Q. When will this algorithm terminate?

A. When every instance is classified correctly.

- That is, there is some weight vector such that
- everything in one class is on one side of $\mathbf{w}^\top \mathbf{x}$
- and everything in the other class is on the other side.

Q. What do we call such data sets? (What is the word for such a data set?)

A. Linearly separable.

Q. What if the data are *not* in fact linearly separable?

A. Then the algorithm will never terminate!

- So in practice, we would set some sort of a limit, a timeout, on this.
- For example, until convergence or until counter reached fixed limit (e.g., 1000)

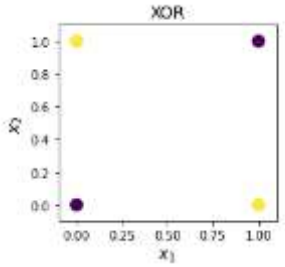
Properties

- Theorem: **converges** if data linearly separable
- If not, then runs forever (or timeout)
- Resulting weight vector cannot be said to minimize anything
- No probabilistic interpretation
- Doesn't generalize well to multiclass

Virtually Died Out …?

- It is true that throughout the 1970s, this whole area of research, though promising, virtually disappeared.
- **Q. Why do you think that happened?**

XOR is not linearly separable



See Jupyter notebook

- The rest of my "notes" are in a Jupyter Notebook on Moodle …
- Experiment in Spring 2020 that didn't work as well as I'd hoped …